

Department of Electronic and Telecommunication Engineering

University of Moratuwa, Sri Lanka

Serial System Bus Design

Multi-Master Shared Bus with Split Transactions
on a Single-Wire Address and Data Path

EN4021 – Advanced Digital Systems

MEMBERS:	INDEX No
Jayakody J.A.K.	220247J
Attanayake A.Y.	220051D
Rajinthan R.	220502M

Target board: Terasic DE2-115 (Intel Cyclone IV E, EP4CE115F29C7)

September 2026

Contents

1	Scope and summary	2
2	Requirements and constraints	2
3	Decisions taken before writing RTL	2
4	What serial signalling implies	3
5	The serial bus	3
5.1	Planning the nets	3
5.2	Eight wires	4
5.3	Inside system_bus	4
5.4	The frame, and the one choice that made it cheap	5
5.5	Decode has to happen after the frame	6
5.6	The data wire needs no turnaround	7
5.7	The return select had to become a latch	7
6	Arbitration	7
7	Split transactions	8
8	The bus must not hang	9
9	Module partitioning	9
10	Verification	10
10.1	Defects found, and where they were	10
11	Results	11
11.1	Wire census	11
11.2	Latency	11
11.3	Implementation	12
12	Limitations and further work	12
13	Conclusion	13
A	Module inventory	13
B	Address map	13

1 Scope and summary

This report covers the design of the **bus** — arbitration, address decoding, the shared signalling wires, the transfer protocol, the split mechanism, and the guarantee that the bus cannot hang. The masters and the memory slaves are treated as endpoints: they appear where they define or constrain the bus contract, and not otherwise.

The headline result is a shared bus in which **the address and the data each travel on a single wire**. The shared segment is eight wires and the whole interconnect is sixty nets. The cost is time — a write takes 21 clocks and a read 30, of which sixteen are the address going out one bit per clock — and the design closes timing at 141.8 MHz against a 50 MHz requirement.

Table 1: Summary of the design. All figures measured, not estimated.

Measure	Result
Shared bus wires	8
Total interconnect nets	60
Write latency	21 clk
Read latency	30 clk
Split read latency (contended)	79 clk
Logic elements	688
Registers	513
$F_{\max}$	141.8 MHz

2 Requirements and constraints

The bus had to support two masters and three memory slaves of 4 KB, 4 KB and 2 KB, with priority arbitration, address decoding, and AHB-style split transactions. Four constraints shaped every decision that follows.

One clock domain, one asynchronous reset.

No gated clocks, no derived clocks, no second domain anywhere in the interconnect. This later ruled out the obvious way of slowing the board demo down (§12).

The bus must never hang.

An access to an unmapped address had to complete, not stall. This is a property of the bus, and it drove a specific structural decision (§8).

Parameterised for three requesters.

A later phase adds a bridge to a second board as a third bus master, so the arbiter could not be hard-coded for two.

The split mechanism written once.

The same phase reuses it for the remote bridge, which will be slow for exactly the same reason a busy slave is.

A 16-bit word address was fixed by the address map: three slaves at 0x0000, 0x1000 and 0x2000, with `addr[15]` reserved for the future remote window and deliberately left unmapped.

3 Decisions taken before writing RTL

The specification left several bus-level questions open. Fixing them first, in writing, was worth the half hour it took: each one is a contract that several modules must agree on, and discovering a disagreement in simulation is far more expensive than deciding it up front.

Table 2: Bus-level decisions and their reasoning.

Question	Decision	Reasoning
Response encoding	OKAY=00, ERROR=01, SPLIT=10	AHB-flavoured. OKAY is all-zero, so a reset or an idle bus reads as “nothing wrong”.
Handshake polarity	active high, except rst_n	One rule, with no exceptions to remember.
Data width	8 bits	On a serial bus this is a direct latency cost — one clock per bit on every transfer. It also makes the slaves 4 KB/4 KB/2 KB, the natural reading of the sizes given.
Retry policy on ERROR	report, never retry	An unmapped address will not become mapped. A retry is an infinite loop holding the bus.
Split visibility	invisible to the requester	One command in, one completion out, however many times the transfer was actually driven. The replay is the bus’s business.

4 What serial signalling implies

The requirement is simple to state and expensive to satisfy: **the address travels on one wire and the data on one wire**, shared by every master and every slave.

Three consequences follow immediately, and each is a design problem rather than a coding detail.

1. **A 16-bit address costs 16 clocks**, on every transfer, unavoidably. Latency stops being a rounding error and becomes the dominant characteristic of the bus.
2. **No slave knows whether it is the target until the address has arrived.** Selection cannot be combinational on a whole address, because there is no whole address to look at — only a bit per clock. The decoder has to watch the address arrive, and the select can only be acted on once it has.
3. **The reply is not at a fixed offset from the address.** A write can be answered immediately; a read cannot answer until it has shifted its data out. Any return path that assumes a fixed reply time breaks, so the routing of the reply has to be held rather than timed.

The interesting engineering is in points 2 and 3. Point 1 is simply the price.

5 The serial bus

5.1 Planning the nets

Before any block diagram is worth drawing, it is worth being precise about what a “shared wire” actually is on this device, because that single fact decides the shape of the whole interconnect.

A shared bus on an FPGA is a multiplexer. The textbook shared bus is wired-OR or tri-stated: every master drives the same physical conductor and only one enables its driver at a time. **A Cyclone IV E has no internal tri-state buffers.** Quartus will accept an inout on an internal net and then silently convert it into exactly the multiplexer one would have written by hand, so there is nothing to be gained by pretending otherwise. The design therefore has no inout ports and no z values anywhere. “Sharing” means two multiplexers:

- a **forward** multiplexer selected by the arbiter’s one-hot grant, putting the granted master’s signals onto the bus;
- a **return** multiplexer selected by the decoder’s select, putting the responding slave’s reply back.

Every net is therefore either private or shared. Each endpoint has its own private stub running to the multiplexer; the shared segment exists only downstream of it. Both are worth counting separately, because they scale differently: private stubs grow with the number of endpoints, while the shared segment is the number that every future endpoint has to tap and route to.

Serialising the payload collapses both, because no stub ever has to carry a whole word. A master's private stub is one address bit, one data bit and two control lines; a slave's reply stub is one data bit plus its ready and response lines. Every payload net in the interconnect is one bit wide, and the width of the address and the data no longer appears in the net count at all — it appears in the clock count instead.

Table 3: Net plan. Widths are per endpoint; the shared segment is what every endpoint taps.

Segment	Width	Units	Carries
Master → forward mux	4	2	m_astream, m_dstream, m_valid, m_we
Shared bus	8	1	the two payload wires and six control wires — see Table 4
Slave → return mux	4	4	s_dstream, s_ready, s_resp [2]
Request / grant	4	1	m_req [2], m_gnt [2]

Two properties of that plan are worth stating in advance, because the rest of the design is built to preserve them:

1. **The shared segment does not grow with the number of slaves.** Adding a fourth slave adds a four-bit private stub and one select line; the eight shared wires are unchanged. That is the property a serial bus is bought for.
2. **No parallel address exists anywhere in the datapath.** If any module reassembled the 16-bit address to decode it, sixteen bits of routing would reappear inside the bus and the saving would be partially undone. The decoder is therefore serial too (§5.5), and the only 16-bit reassembly in the design is a debug observation point feeding the JTAG probe, removable by a parameter.

5.2 Eight wires

Table 4: The complete shared bus. Everything else is point-to-point.

Wire	Width	Role
bus_astream	1	The address. Driven by the granted master; received by every slave and by the decoder's deserialiser.
bus_dstream	1	The data. Half duplex.
bus_valid	1	Frame marker, high for ADDR_W clocks.
bus_we	1	Direction; also the data wire's direction control.
bus_ready	1	Completion strobe.
bus_resp	2	OKAY / ERROR / SPLIT.
master_id	1	Tag of the granted master, so a split-capable slave can record whose transfer it deferred.

5.3 Inside system_bus

system_bus is the interconnect and nothing else. It contains four blocks and one debug observation point, and it deliberately contains no master and no memory — those are endpoints, instantiated outside it and reaching it only through the serial ports above. The module can therefore be tested on its own, with nothing attached at either end, which is what makes the sharpest test in the suite possible (§8).

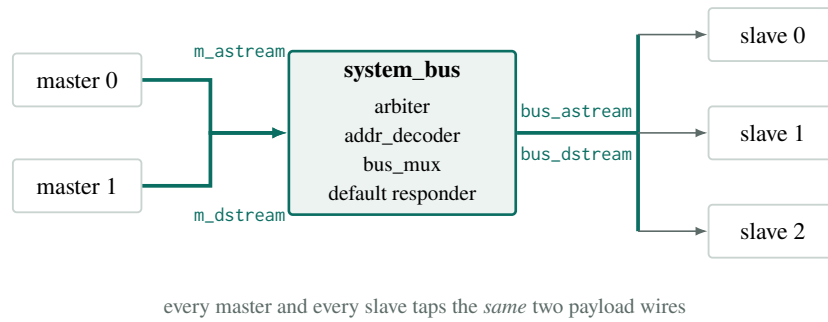


Figure 1: Bus topology. The shared segment is two payload wires plus six control wires; every per-endpoint stub is one bit.

Table 5: The four blocks inside system_bus.

Block	Responsibility
arbiter	Decides who owns the bus. Fixed priority by index, a lock that holds the grant until the transfer completes, and a split mask that removes a deferred master from arbitration entirely. It never sees the serial wires — only req, bus_ready and bus_resp — which is why serialising the payload did not change it.
addr_decoder	Watches the address arrive bit by bit and narrows which slave can still match. Emits a one-hot select, or the default select if nothing matched. Serial, not a comparator behind a deserialiser (§5.5).
bus_mux	The datapath. The forward multiplexer puts the granted master onto bus_astream, bus_valid and bus_we; the return multiplexer routes the responding slave's ready, resp and data back. It also owns the direction of the one shared data wire.
default_slave	Answers any address the decoder did not map, with ERROR, on the normal one-cycle schedule. It is the reason the bus cannot hang, and it lives here rather than outside because that is a property of the interconnect (§8).

How a transfer flows through it. A master raises m_req. The arbiter registers a one-hot m_gnt and locks. The granted master then drives its frame, and bus_mux forwards it onto the shared wires, where every slave and the decoder see it simultaneously. When the frame falls, addr_done strobes the decoder's standing answer out as a one-cycle select; bus_mux captures that select and *holds* it (§5.7). Whichever responder was selected eventually raises ready, the held select routes it back to bus_ready and bus_resp, the arbiter unlocks, and the master completes. Exactly one responder is ever selected, and exactly one transfer is ever outstanding.

What is deliberately *not* in here. No memory, no master, and no parallel address. The 16-bit deserialiser that does exist in system_bus feeds the JTAG probe only; nothing in the datapath reads it, and setting OBSERVE_ADDR=0 removes it along with the last parallel address anywhere in the design. The whole regression still passes with it removed, apart from the four checks that read the probe itself — which is the standing test that the datapath really is serial.

5.4 The frame, and the one choice that made it cheap

An address phase is ADDR_W=16 clocks with bus_valid high. The address goes out most-significant bit first. On a write, the data goes out on the other wire *at the same time* — and **right-aligned**.

Right-aligning the data is the single most useful decision in the design. Because the data sits at the *end* of the frame, every receiver can be a plain shift register of its own width, clocked for the whole frame, holding *the last W bits it saw* — and that is exactly the field it wanted.

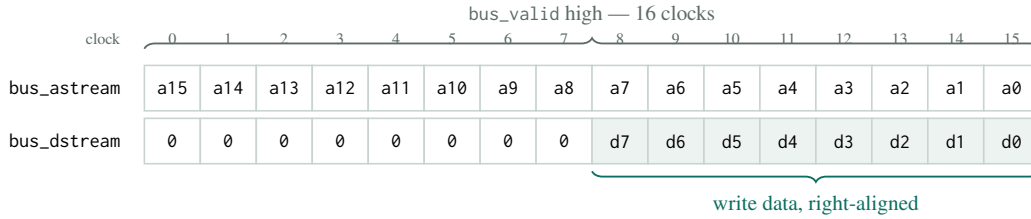


Figure 2: The address frame. Padding the write data with leading zeros costs nothing and removes every bit counter from the receiving side.

Table 6: Four receivers, one wire, no bit counters.

Receiver	Width	Ends up holding	Discards
Central deserialiser (for the decoder)	16	the whole address	—
Slave 0 / slave 1 address	12	addr[11:0], its offset	addr[15:12]
Slave 2 address	11	addr[10:0], its offset	addr[15:11]
Any slave's write data	8	the data byte	the padding

The upper address bits shift straight through a slave's narrow register and fall off the end; deciding *which* slave is the decoder's job, and it is the only receiver that needs all sixteen bits. The result is that **there is exactly one frame timer on the entire bus** — a counter inside the granted master — and every other endpoint simply follows bus_valid. The serialiser makes the padding automatic by shifting zeros in behind the data:

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) sr <= {W{1'b0}};
    else if (load) sr <= din;
    else if (shift) sr <= {sr[W-2:0], 1'b0}; // zeros in behind
end
assign dout = sr[W-1];
```

5.5 Decode has to happen after the frame

Since no slave knows whether it is selected until the last address bit arrives, **every slave shifts every frame in, addressed or not**, and selection arrives afterwards as a one-cycle pulse. Shifting speculatively costs nothing: a slave that was not selected never acts on what it collected. The corresponding hazard — a slave acting on a frame without a select, and corrupting another slave's transfer — is tested directly (§10).

The decode strobe is derived rather than transmitted:

```
reg bus_valid_d;
always @(posedge clk or negedge rst_n)
    if (!rst_n) bus_valid_d <= 1'b0;
    else bus_valid_d <= bus_valid;

assign addr_done = bus_valid_d & ~bus_valid; // falling edge of the frame
```

Using the falling edge of bus_valid rather than adding an “end of frame” wire keeps the shared bus at eight wires and — more usefully — leaves the frame length defined in exactly one place. Nothing else on the bus knows or cares how long a frame is.

The decoder itself does not wait for the frame to end, though. It is a **progressive prefix matcher**: a one-hot position marker walks the first PFX_W bits of the frame and one *alive* bit per slave is cleared the moment the wire disagrees with that slave's prefix. Because every slave occupies a power-of-two block, its address range *is* a prefix, and the answer is settled after five bits — eleven clocks before the

frame ends. `addr_done` merely strobes a result that has been standing for some time. The cost is eight flip-flops, and no 16-bit address is ever assembled to compare against.

5.6 The data wire needs no turnaround

A half-duplex wire normally needs a turnaround cycle and some way to stop two drivers overlapping. This one does not, because the two directions cannot overlap by construction:

	During the frame	After the frame
Write	master sends	—
Read	—	slave sends

So the direction control is nothing more than `bus_we`, which the granted master holds stable for the whole transaction:

```
assign bus_dstream = bus_we ? m_dstream_sel : s_dstream_sel;
```

The master explicitly releases the wire on a read, and each slave drives only while actually shifting data out. Both are belt-and-braces given the multiplexer above, but they keep an idle bus readable in a waveform, and a testbench asserts that the master never drives during a read.

5.7 The return select had to become a latch

The obvious way to steer the return multiplexer is a one-cycle-delayed copy of the decoder select — a single register — and it looks correct until one notices that it assumes the reply arrives at a fixed offset. On this bus it does not:

Table 7: Completion timing, measured from the select pulse *S*.

Case	Ready at	Why
Write	$S+1$	the word is committed at S
SPLIT	$S+1$	the slave is getting out of the way
Unmapped	$S+1$	the default responder answers at once
Read	$S+10$	memory access, then eight bits shifted out

A one-cycle-delayed select goes stale nine cycles before a read reply arrives. The select is therefore *captured* on the decoder's pulse and *held* until the next transfer selects something else. That is safe precisely because the arbiter locks the bus: only one transfer is ever outstanding.

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) sel_q <= {(N_SLAVES+1){1'b0}};
    else if (!sel) sel_q <= sel;           // capture and HOLD
end
```

6 Arbitration

Fixed priority, master 0 highest, with two mechanisms layered on top.

Bus lock. Once a master is granted, the grant is held until that transfer completes, so a transfer is never torn in half. The arbiter re-arbitrates on the cycle after completion, which stops a high-priority master owning the bus across a whole burst while keeping each transfer atomic. This matters far more after serialisation: the lock now spans 21 to 30 clocks rather than 5, so a lock that released early would open a much larger window of corruption.

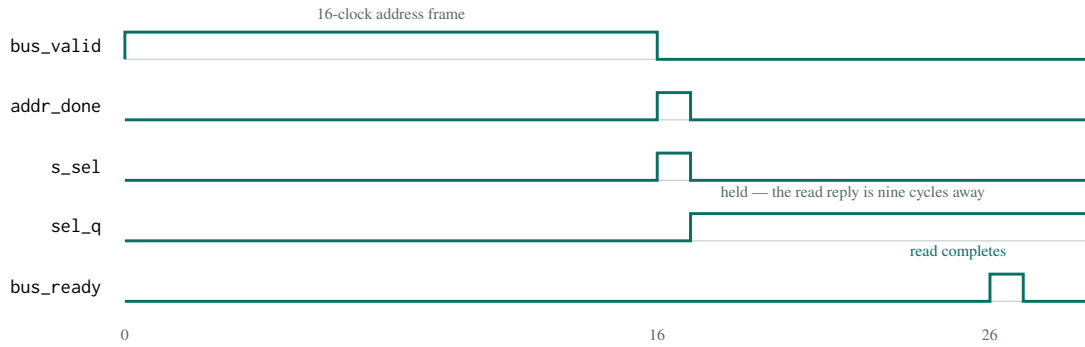


Figure 3: A read. The select is a one-cycle pulse; the *latched* copy is what routes the reply nine cycles later.

Split mask. One bit per master. A master whose transfer is answered SPLIT has its mask bit set and is removed from arbitration entirely; the bit is cleared when the slave signals completion. Crucially, **the deferred master keeps its request asserted the whole time**. Masking, not request withdrawal, is what defers it. That keeps the decision about *when* to replay inside the master and leaves the arbiter a pure scheduler.

Parameterisation. N_MASTERS and ID_W are parameters, the priority encoder is a loop, and the mask is a per-bit vector, so the phase-2 bridge joins as requester 2 without touching the logic. The arbiter is also the one module that **did not change at all** when the bus was serialised: it only ever observed req, bus_ready and bus_resp, none of which are serial.

7 Split transactions

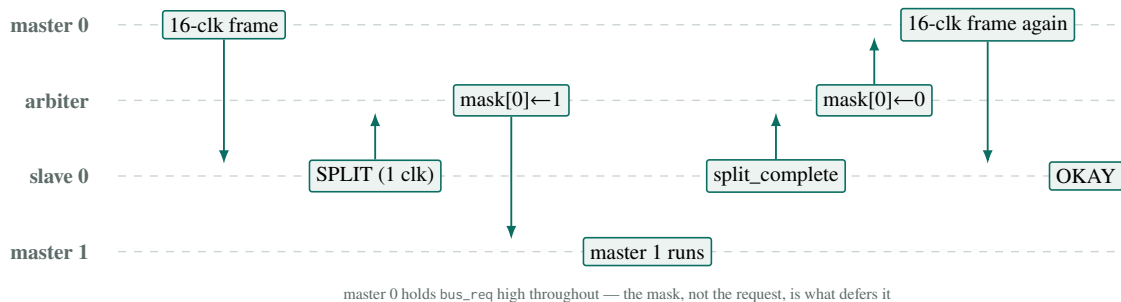


Figure 4: A split transaction. The replay re-transmits the complete frame; it is a re-issue, not a resumption.

Three properties define the mechanism at bus level.

The deferred transfer is not performed.

No write lands, no read happens. Since the master re-transmits the entire frame, the slave only has to remember *who* it deferred — one master ID — and no address or data storage is needed anywhere in the split logic.

The replay is a complete frame,

not a resumption of a half-finished one. This is verified explicitly: a split transaction must put *two* full-length frames on the wire.

One split outstanding.

An access arriving while the slave is busy, or while a resume is still owed, is served normally. This is a deliberate simplification: with single-entry deferred state, allowing a second split would mask two masters with only one wake-up available to release them — a deadlock.

Why splitting earns its keep here. The asymmetry is what makes the mechanism worth its cost. A slave answers SPLIT in **one** clock, while a real read costs **ten** and the whole transaction costs thirty. Deferring therefore gets the slow slave out of the way almost immediately and hands the other master a genuinely useful window — tens of clocks, not a handful. On a bus where every transfer is dominated by a 16-clock address phase, that window is long enough for the other master to complete entire transactions inside it, which is exactly what the integration testbench demonstrates.

8 The bus must not hang

Without a responder for unmapped addresses, an access to a decode hole asserts no slave select, nothing ever drives bus_ready, and the granted master waits forever holding the bus. Nothing but a reset recovers it. This is an easy fault to create by accident: it is enough for a decoder to generate a “no slave matched” signal and for the interconnect to leave that signal unconnected, at which point the hole is invisible until something addresses it.

The fix is a **default responder** that answers any unmapped address with ERROR on the normal one-cycle schedule. It needs no deserialisers — it does not care what the address or the data were, only that nothing else claimed them — and it never drives the data wire, so a read of an unmapped address reassembles zero.

It belongs inside the bus. This was the one genuine judgement call in the module partitioning. The default responder looks like a slave and was named like one, but it holds none of the state a peripheral has — no memory, no address of its own — and its entire purpose is to stop an unmapped address leaving the bus without a responder. That is a property of the interconnect, not of anything hanging off it. Placing it inside system_bus also means N_SLAVES counts only real slaves, and it makes possible the sharpest test in the suite: *an unmapped frame is answered ERROR by the bus with nothing whatsoever attached to its slave ports.*

9 Module partitioning

The design is three kinds of module, and the split is strict.

Table 8: What each module may contain.

Module	Contains	Must not contain
system_bus	arbiter, address decoder, bus_mux, the central address deserialiser, the default responder	any master, any memory
master	command FSM, serialisers, read deserialiser, replay	arbitration, decoding
slave	memory, deserialisers, output shift register, split state	arbitration, decoding

There is no integration wrapper. de2_top instantiates the three side by side, along with the one OR gate that combines the split wake-ups per master. That was a deliberate call and it has a cost: the same composition appears in de2_top and in both integration testbenches, which cannot instantiate a wrapper that does not exist. Change the wiring in one and the others must follow. A wrapper would remove that duplication at the price of an extra level of hierarchy; the hierarchy was judged the greater cost.

Both interfaces are serial — nothing wide crosses either boundary.

bus_dstream is exposed once from system_bus and wired to both masters and slaves, so the shared data wire is literally one signal in the netlist rather than a pair that happens to be connected. Each endpoint has an output and an input onto that net rather than one bidirectional port, for the reason given in §5.1: there are no internal tri-states to use.

Table 9: The two interfaces. Per-endpoint streams are *candidates* into the multiplexer; the bus_ wires are single nets every endpoint taps.

Interface	Address	Data
master → bus	m_istream, 1 per master	m_dstream, 1 per master
bus → slave	bus_istream, 1 shared	bus_dstream, 1 shared
slave → bus	—	s_dstream, 1 per slave
bus → master	—	bus_dstream, the same net

The partitioning was done *after* the bus was working, and it cost nothing — the build before and after is identical to the logic element. That is the expected result for a pure refactor, and confirming it against the fitter was worth the two minutes.

10 Verification

Thirteen self-checking testbenches carrying 451 individual checks, each printing its own PASS / FAIL banner and an error count, driven by a script that greps the output and sets the exit status — the testbenches themselves always exit zero, so the exit code cannot be trusted.

Two decisions were worth more than the rest.

Every wait has a cycle budget. A hang is reported as a FAILURE, not endured until the simulator gives up. On a bus whose central guarantee is that it cannot hang, a test that hangs is a test that found the bug and then failed to report it.

The bus is tested with nothing attached. tb_system_bus instantiates no master and no memory. It plays both roles by driving the two serial interfaces directly: shifting an address frame in on a master's wire, answering as a slave on a slave's wire. This is possible only because of the partitioning, and it makes bus-level properties directly observable rather than inferred from system behaviour.

Table 10: Bus properties and where each is proven.

Property	Testbench
Every frame is exactly ADDR_W clocks, including a replayed one	tb_integration, tb_master
The address reassembled off one wire matches what was sent	tb_system_bus, tb_master
Write data arrives right-aligned	tb_master
Bit order is MSB-first (0x80 and 0x01 both round-trip)	tb_master, tb_shift_ser
A slave ignores a frame it was not selected for	tb_slave
Only the low address bits reach a slave	tb_slave, tb_shift_deser
The data wire's direction is exactly bus_we, and neither side can disturb the other	tb_system_bus, tb_bus_mux
A reply arriving ten cycles late still routes from the right slave	tb_system_bus, tb_bus_mux
A masked master is excluded while still requesting	tb_arbiter, tb_system_bus
A split puts <i>two</i> complete frames on the wire	tb_integration, tb_master
Exactly one responder for all 65,536 addresses	tb_addr_decoder
An unmapped frame is answered ERROR <i>by the bus alone</i>	tb_system_bus
Four consecutive bad addresses do not wedge the bus	tb_integration

10.1 Defects found, and where they were

Worth recording honestly, because the distribution is informative: of the significant defects found during bring-up, **none were in the bus protocol itself**, and several were in the tooling around it.

- **A counter narrower than its own parameter.** The slave’s busy counter was 16 bits while the board passes a split latency of 10,000,000 clocks. The constant was silently truncated to 38,528 — the split would have been 260× shorter than intended on hardware, and nobody would have noticed. Now 32 bits.
- **Read-during-write forcing an undefined memory.** Reading and writing the array in the same cycle makes Quartus infer a RAM whose read-during-write result it explicitly documents as undefined; it would not have matched simulation. The enables are now mutually exclusive.
- **The fitter deleting what cannot be observed.** Twice: read data that only partly reached a pin, and a test pattern with two identical byte lanes. Both produced memories narrower than the design specifies. The lesson generalises — *a datapath the fitter cannot see reaching an output is a datapath it will not build* — and total memory bits in the fit report is now a checked number.
- **Verilog tasks are static by default.** A testbench called the same task from two branches of a fork; the two calls shared storage and corrupted each other. Three tests failed for a reason unrelated to the RTL.
- **A comment parsed as a pragma.** A comment whose text began `// synthesis` in each instance produced three “unrecognized synthesis attribute” warnings.

11 Results

11.1 Wire census

The interconnect as built, counted from the netlist. The shared segment is what every endpoint has to tap; the private stubs are one bit each because every payload signal on this bus is one bit wide.

Table 11: Serial bus net count.

Segment	Per unit	Units	Nets
Master → forward mux (private)	4	2	8
Shared bus	8	1	8
Slave → return mux (private)	4	4	16
Arbitration and select	11	1	11
Decode path inside system_bus	17	1	17
Total			60

The eight shared wires are the number that matters: adding a fourth slave adds four private nets and changes the shared segment not at all.

11.2 Latency

All figures measured in simulation, from command acceptance to completion.

Table 12: Transaction cost. The nine-clock gap between a write and a read is exactly the read data phase.

Case	Clocks
Write	21
Read	30
Read, contended (winner / loser)	30 / 59
Split read (SPLIT_LATENCY=6), uncontended	57
Split read (SPLIT_LATENCY=6), other master contending	79

Sixteen of a read's thirty clocks are the address going out one bit at a time. That is the dominant term, and it is the price of a one-wire address.

11.3 Implementation

Table 13: Quartus Prime Lite 24.1std, EP4CE115F29C7, full map / fit / sta / asm flow.

Measure	Result
Errors	0
Inferred latches	0
Combinational loops	0
Logic elements	688
Registers	513
Memory bits (all M9K)	81,920
$F_{\max}$ (slow, 1200 mV, 85 °C)	141.8 MHz

Zero inferred latches and zero combinational loops are not incidental: every combinational block in the design assigns a default to each of its outputs before the case statement that may override it, which is what makes the latch count structurally zero rather than luckily zero.

The $F_{\max}$ is worth a note. The widest combinational path on the bus is the return multiplexer, and on this design it is *one bit* wide — the reply is a single serial wire plus a two-bit response code. Serialising the payload does not merely save wires; it shortens the critical path, which is why a bus that costs six times the clocks closes timing with a 2.8× margin on the 50 MHz requirement.

12 Limitations and further work

Fixed priority can starve the low-priority master.

With master 0 requesting continuously, master 1 never runs. The bus lock bounds the wait to one transfer, but priority is priority. Round-robin or a weighted scheme would fix it; the arbiter's structure supports either without touching anything else.

The address frame dominates latency.

Sixteen of thirty clocks carry an address that is usually close to the previous one. Two obvious improvements: pipeline the next frame against the current data phase, or send a short frame when the upper address bits are unchanged. Both were out of scope; both are worth more than any other optimisation available.

One split outstanding.

A second split-capable slave, or a second outstanding split on slave 0, would need per-master deferred state and one wake-up per entry.

Board demo throttling, not clock division.

The specification asked for a PLL or clock divider to slow the demo down. A divided clock is a derived clock and would violate the project's own one-domain, no-gated-clock rule, so the bus runs at 50 MHz and only the scenario sequencer is throttled by a slow clock-enable tick. Same visible effect, one clock domain, fully timing-analysable.

Memory contents are undefined at power-up.

The arrays are deliberately unreset — an asynchronous reset on a 4096-word array stops M9K inference and builds the memory from flip-flops instead — so every test writes a location before reading it.

Phase 2 readiness. The remote bridge joins as a third bus requester. Three things were arranged in advance: the arbiter is parameterised on `N_MASTERS` and already exercises splits on both a high- and a low-priority master; the split mechanism is written once and is enabled by a single parameter on any slave; and `addr[15]=1` is reserved and unmapped, with an exhaustive 64K decoder sweep that fails if anything is ever mapped there.

13 Conclusion

The bus does what was asked: two masters, three slaves, priority arbitration with a bus lock, address decoding with a guaranteed responder, and split transactions — with the address and the data each carried on a single shared wire.

The engineering judgement that mattered most was not any of the protocol mechanisms; it was **right-aligning the write data inside the address frame** (§5.4). That one choice turns every receiver on the bus into a plain shift register holding “the last *W* bits I saw”, removes bit counters from four different modules, and leaves exactly one frame timer in the entire system. It cost eight cycles of padding on a write — cycles the address phase was going to spend anyway — and bought a bus with no distributed timing state at all.

The second was recognising that serialisation changes which mechanisms are worth having, not just how fast the bus runs. When a slave can decline a transfer in one clock but serving it costs thirty, the split mechanism stops being a specification box to tick and becomes the part of the design doing the most work — and, for the same reason, the default responder stops being a nicety and becomes the only thing standing between an unmapped address and a permanently wedged bus.

A Module inventory

Module	Role	Notes
system_bus	the bus	arbiter, decoder, mux, address deserialiser, default responder
arbiter	scheduling	fixed priority, bus lock, split mask; parameterised on <code>N_MASTERS</code>
addr_decoder	decode	purely combinational, one-hot + default
bus_mux	data path	who drives the two shared wires; latched return select
shift_ser	primitive	parallel in, serial out; pads with zeros
shift_deser	primitive	serial in, parallel out; holds the last <i>W</i> bits
default_slave	bus service	answers <code>ERROR</code> for unmapped addresses
master	endpoint	serialises, replays after <code>SPLIT</code>
slave	endpoint	memory; <code>SPLIT_CAPABLE</code> adds split state
de2_top	board	clocking, buttons, switches, displays, and the composition of the three above
bus_issp_driver	debug	JTAG front-end, ISSP instance “SBUS”

B Address map

Range	Responder	Size	Decode
0x0000–0x0FFF	slave 0	4 KB	<code>addr[15:12]==4’h0</code>
0x1000–0x1FFF	slave 1	4 KB	<code>addr[15:12]==4’h1</code>
0x2000–0x27FF	slave 2	2 KB	<code>addr[15:11]==5’b00100</code>
0x2800–0x2FFF	default	—	decode hole
0x3000–0x7FFF	default	—	unmapped
0x8000–0xFFFF	default	—	reserved , phase-2 remote window

Slave 0 is the split-capable one. Every unmapped range answers ERROR on the normal one-cycle schedule, so no address in the 64K space can stall the bus.